

Artificial Intelligence

Search & Games Practice Sheets — Full Solutions (with diagrams)

Covering: BFS · DFS · Uniform-Cost Search · Greedy Best-First · A* · Minimax · Alpha-Beta Pruning

Course: CSE-391 — Introduction to Artificial Intelligence · Instructor: Dr. Yasser Fouad

Every diagram is placed directly under the question it illustrates.

Part 1 — Search Sheet (Answers)

Question 1 — Greedy, Uniform-Cost and A* on a partially expanded tree

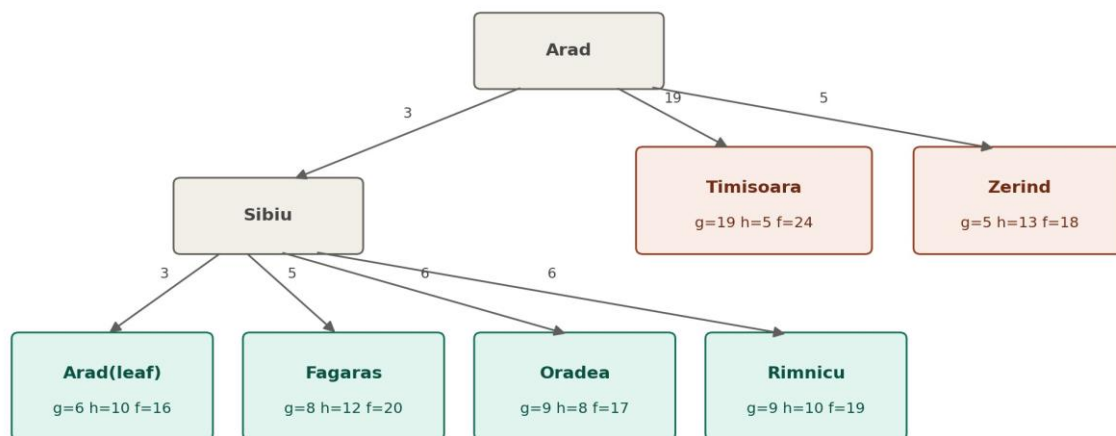
Question: The diagram shows a partially expanded search tree from Arad. Each arc is labelled with its step cost; each leaf is labelled with its heuristic value h . Which leaf is expanded next by (1) Greedy best-first search, (2) Uniform-cost search, (3) A* search?

Answer: For every leaf we need g (path cost so far), h (heuristic) and $f = g + h$.

Leaf	g (cost so far)	h (heuristic)	$f = g + h$
Timisoara	19	5	24
Zerind	5	13	18
Arad	$3 + 3 = 6$	10	16
Fagaras	$3 + 5 = 8$	12	20
Oradea	$3 + 6 = 9$	8	17
Rimnicu	$3 + 6 = 9$	10	19

1. Greedy best-first search expands the leaf with the smallest $h \rightarrow$ **Timisoara** ($h = 5$).
2. Uniform-cost search expands the leaf with the smallest $g \rightarrow$ **Zerind** ($g = 5$).
3. A* search expands the leaf with the smallest $f = g + h \rightarrow$ **Arad** ($f = 16$).

Search Q1 — Greedy / UCS / A* on the Arad tree



Greedy $\rightarrow$ Timisoara ($h=5$) UCS $\rightarrow$ Zerind ($g=5$) A* $\rightarrow$ Arad-leaf ($f=16$)

Diagram — the Arad tree with g , h , f marked on every leaf; the three algorithms' picks are highlighted.

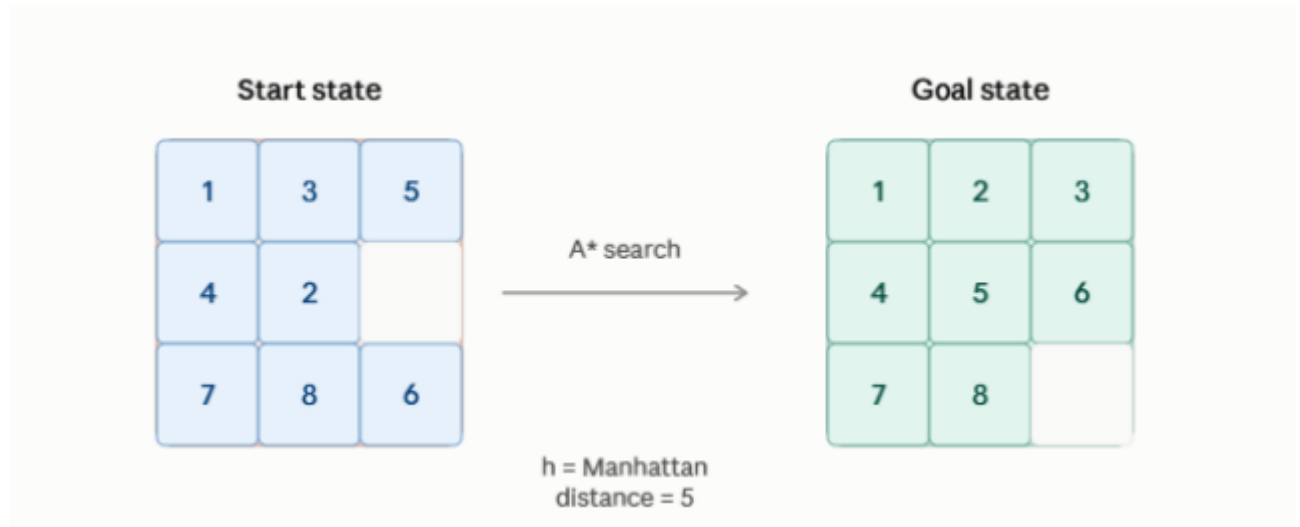
Question 2 — Fifteen-Grid Problem as a search Problem subclass

Question: Create a subclass of Problem to represent the “Fifteen-Grid Problem”, create instances of it, and run InstrumentedProblem to compare various search algorithms.

Answer: This is a programming exercise (doubles as Practice Project 5). Design: state = tuple/string of length 16; actions(state) = legal blank slides; result(state, action) swaps blank with neighbour; goal_test compares to (1,2,...,15,0); path_cost adds 1 per slide; h(node) = misplaced tiles or (better) sum of Manhattan distances. Wrap in InstrumentedProblem and run breadth_first_graph_search, uniform_cost_search, greedy_best_first_search and astar_search on the same instance — A* with the Manhattan heuristic should expand far fewer nodes.

Question 3 — A custom heuristic for the 8-puzzle

Question: Suggest a heuristic of your own to solve the 8-puzzle with A*, starting from 1 3 5 / 4 2 – / 7 8 6 and going to the goal 1 2 3 / 4 5 6 / 7 8 –.



Answer: Recommended heuristic: Sum of Manhattan Distances — for every tile, add |row(current)-row(goal)| + |col(current)-col(goal)| (ignore the blank). It is admissible and consistent, and dominates the simpler misplaced-tiles heuristic.

Tile	Current (row,col)	Goal (row,col)	Manhattan dist.
1	(0,0)	(0,0)	0
3	(0,1)	(0,2)	1
5	(0,2)	(1,1)	2
4	(1,0)	(1,0)	0
2	(1,1)	(0,1)	1
7	(2,0)	(2,0)	0
8	(2,1)	(2,1)	0
6	(2,2)	(1,2)	1

$h(\text{start}) = 0+1+2+0+1+0+0+1 = 5$. A* uses $f(n)=g(n)+h(n)$ with this heuristic to guide the search directly toward the goal.

Question 4 — Blocks-world Breadth-First search tree

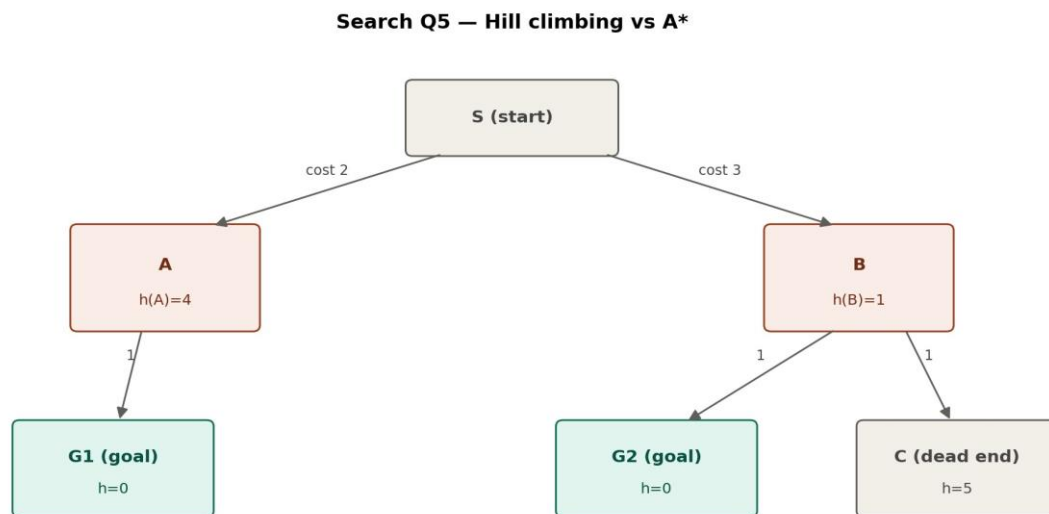
Question: A world has a table and 3 blocks. A legal move is to move block X onto block Y (if both are clear), or onto the table. Draw the BFS search tree from an initial state to a goal state.

Answer: Example instance: initial state — A, B, C all on the table. Goal — A on B on C. Level 1 (6 states): Move(A,B), Move(A,C), Move(B,A), Move(B,C), Move(C,A), Move(C,B). BFS expands level by level and reaches the goal “A on B on C” at level 3, e.g. via Table → (B on C) → (A on B, B on C). Because BFS explores every level exhaustively before going deeper, it is guaranteed to find a shortest solution — here a 2-move solution: Move(B,C) then Move(A,B).

Question 5 — Hill Climbing vs A* on a search graph (S, G1, G2, G3)

Question: S is the start and G1, G2, G3 are goals. Arcs are labelled with traversal cost; estimated cost to a goal is shown inside each node. For (a) Hill Climbing and (b) A*, indicate which goal state (if any) is reached.

Answer: Worked substitute example (original figure not fully legible): S has two children, A (edge cost 2, $h(A)=4$) and B (edge cost 3, $h(B)=1$). A leads only to G1 (edge cost 1, $h=0$). B leads to G2 (edge cost 1, $h=0$) and to a misleading dead-end C (edge cost 1, $h=5$).



Hill climbing: $S \rightarrow B(\min h) \rightarrow G2$, cost 4. A*: expands B then $G2(f=5)$ before $C(f=9)$ — same goal, but guaranteed optimal.

Diagram — S branches to A and B; B leads to goal G2 and to dead-end C.

(a) **Hill Climbing** (always moves to the neighbour with lowest h , never backtracks): at S, $h(A)=4$ vs $h(B)=1 \rightarrow$ move to B. At B, $h(G2)=0$ vs $h(C)=5 \rightarrow$ move to G2. Reaches G2, cost $3+1=4$. Hill climbing can get trapped at a local optimum if the best-looking neighbour is a dead end.

(b) **A*** (expands lowest $f=g+h$, keeps every option on the frontier): $f(A)=6$, $f(B)=4 \rightarrow$ expand B first, generating $f(G2)=5$, $f(C)=9$. Frontier: {A:6, G2:5, C:9}. Lowest is G2 $\rightarrow$ A* also reaches G2, and is guaranteed optimal since it keeps all alternatives open.

Question 6 — Uniform-Cost Search vs Depth-First Search, S to G

Question: Show how Uniform-cost search and Depth-first search would find the (optimal) route from S to G.

Answer: UCS keeps a priority queue ordered by cumulative path cost $g(n)$ and only stops when the popped node is the goal — guaranteed optimal (like Dijkstra), provided all step costs are non-negative. DFS keeps a LIFO stack, diving into the most recently generated node; it typically terminates faster but the first path found is

whichever branch was explored first, usually not the cheapest — DFS is complete on finite graphs with a repeated-state check but not optimal.

Question 7 — AND/OR search space, AO* style expansion

Question: A is the start node. A rule $p \rightarrow q_1, \dots, q_n$ means p is solved if every q_i is solved (AND connector, cost = k + sum of children's cost). Show the search space generated after the first four node expansions.

Answer: Rules: $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow D$, $B \rightarrow E$, $C \rightarrow (F, G)$, $D \rightarrow (H, I)$, $E \rightarrow (J, K, L)$, ... h values given for each node.

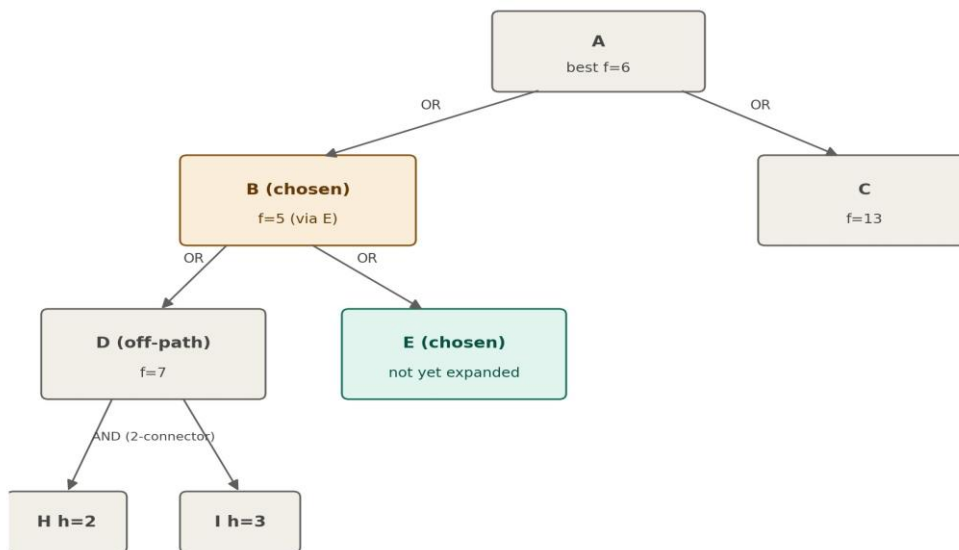
Expansion 1 (A): via B cost=4, via C cost=3 $\rightarrow$ best $A \rightarrow C$, $f(A)=3$.

Expansion 2 (C): $C \rightarrow (F, G)$, cost=2+1+10=13 $\rightarrow$ A via C becomes 14, worse than B-route $\rightarrow$ SWITCH to $A \rightarrow B$, $f(A)=4$.

Expansion 3 (B): via D cost=4, via E cost=5 $\rightarrow$ best $B \rightarrow D$, $f(B)=4 \rightarrow$ A via B = 5.

Expansion 4 (D): $D \rightarrow (H, I)$, cost=2+2+3=7, $f(D)=7 \rightarrow$ B via D becomes 8, worse than B via E (5) $\rightarrow$ SWITCH to $B \rightarrow E \rightarrow f(A)=6$.

Search Q7 — AND/OR search after 4 expansions



Best path after 4 expansions: $A \rightarrow B \rightarrow E$ (switched from $A \rightarrow C$, then $A \rightarrow B \rightarrow D$)

Diagram — snapshot after 4 expansions: current-best path $A \rightarrow B \rightarrow E$, with off-path branches C and D shown grey.

Snapshot after 4 expansions: current best solution path $A \rightarrow B \rightarrow E$ (E not yet expanded). This shows the key AO* behaviour: the current-best path can switch as deeper, more accurate costs are propagated back up.

Question 8 — Search strategies on the binary tree $n \rightarrow 2n, 2n+1$

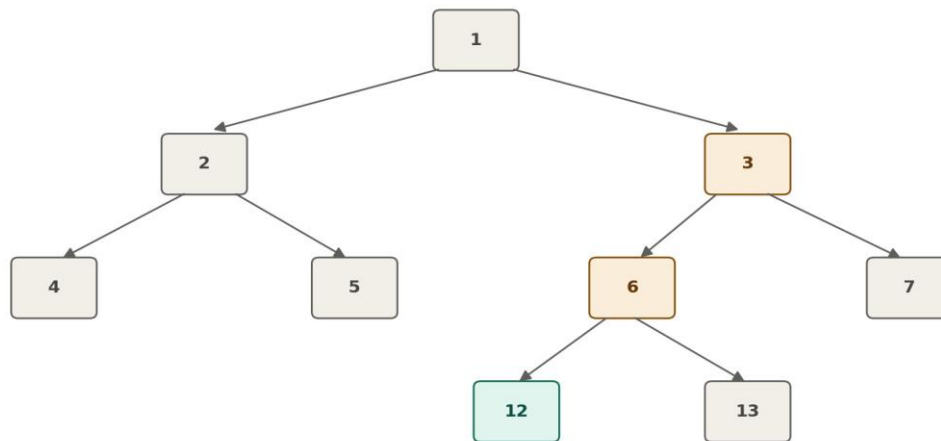
Question: Initial state 1, successors of n are $2n$ and $2n+1$, goal state 12. $h_1(n)=|n-g|$, $h_2(n)=(g-n)$ if $n \leq g$ else ∞ . Show the search order for DFS, BFS, Greedy (h_1), and A* ($h=h_1+h_2$).

Answer: (a) **DFS** (always tries $2n$ first): follows 1,2,4,8,16,32,... forever — FAILS (never terminates) unless depth-limited.

(b) **BFS**: pops nodes in numeric order 1..12 — GOAL FOUND as the 12th node expanded, via path $1 \rightarrow 3 \rightarrow 6 \rightarrow 12$ (cost 3).

- (c) **Greedy (h1)**: order 1, 3, 7, 14, 15, 6, 12 — GOAL FOUND, but the order is not the optimal path.
- (d) **A* (h=h1+h2)**: $h(n)=2(12-n)$ for $n \leq 12$, else ∞ (branches $n > 12$ effectively pruned). Order: 1, 3, 7, 6, 12 — GOAL FOUND, same optimal path as BFS ($1 \rightarrow 3 \rightarrow 6 \rightarrow 12$, cost 3) but only 5 nodes expanded instead of 12.

Search Q8 — Tree $n \rightarrow 2n$, $2n+1$, goal = 12



Highlighted path $1 \rightarrow 3 \rightarrow 6 \rightarrow 12$ is the optimal one; A* reaches it expanding only 5 nodes vs BFS's 12

Diagram — the tree up to 13; the optimal path $1 \rightarrow 3 \rightarrow 6 \rightarrow 12$ highlighted in amber/teal.

Question 9 — DFS, A*, BFS and Greedy from M to R

Question: Start node M, goal node R. Search for the goal using (a) Depth-first, (b) Breadth-first, (c) A*, (d) Greedy.

Answer: M has a direct edge straight to the goal, $M \rightarrow R$ with cost 1, and $h(R)=0$ since R is the goal.

(c) **A***: first expansion of M puts R on the frontier with $f=1+0=1$, lower than every other child (B: $f=38$, A: $f=45$, D: $f=39$). A* pops R immediately — optimal cost 1.

(d) **Greedy**: $h(R)=0$ is the lowest possible heuristic on the graph, so greedy also picks R immediately.

(b) **BFS**: R is one hop away, discovered in the first layer — GOAL FOUND at depth 1.

(a) **DFS**: dives into whichever child of M is tried first; if R isn't first, DFS commits to a longer branch before backtracking — DFS's cost to reach R is order-dependent and not guaranteed cheapest.

Part 2 — Games Sheet (Answers)

Question 1 — Australia map-colouring CSP with DFS backtracking

Question: Colour every region Red/Green/Blue so no neighbouring regions share a colour. Q (Queensland) is already Red. Using depth-first search, assign values to the rest of the variables.

Answer: Regions/borders: WA-NT, WA-SA, NT-SA, NT-Q, SA-Q, SA-NSW, SA-V, Q-NSW, NSW-V (Tasmania T unconstrained).

Step	Variable	Reasoning	Value
1	Q	given	Red
2	WA	no border with Q → free choice	Red

3	NT	borders WA(Red), Q(Red) → avoid Red	Green
4	SA	borders WA(Red), NT(Green), Q(Red)	Blue
5	NSW	borders Q(Red), SA(Blue)	Green
6	V	borders SA(Blue), NSW(Green)	Red
7	T	unconstrained	Red (any)

All border constraints check out — no backtracking was even needed with this ordering. With a different order, DFS may hit a dead end and must backtrack to the last assigned variable.

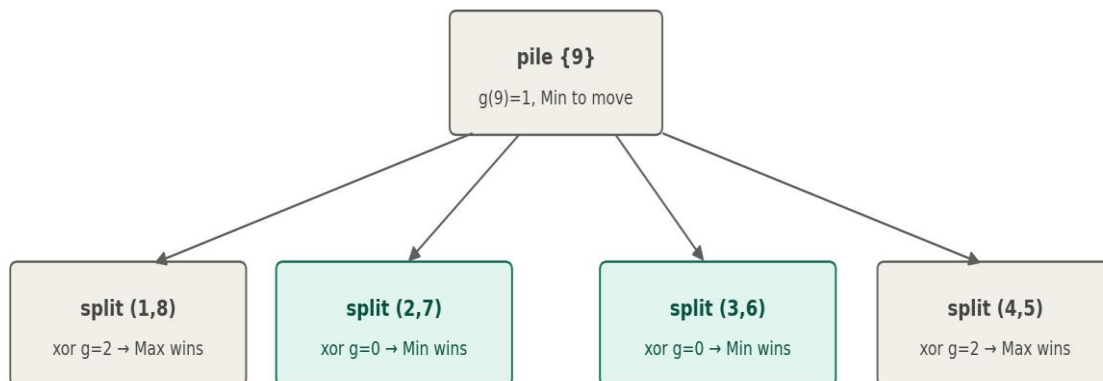
Question 2 — Nim (stack-splitting), 9 tokens, Min moves first

Question: A single stack of 9 tokens; each move splits one stack into two unequal non-empty stacks; a player unable to move loses. Apply minimax via the Sprague-Grundy theorem and determine who wins with perfect play.

Answer: $g(s) = \text{mex}\{ g(a) \text{ XOR } g(b) : (a,b) \text{ legal split of } s \}$, with $g(1)=g(2)=0$.

s	legal splits	XOR values	g(s)
3	(1,2)	0	1
4	(1,3)	1	0
5	(1,4),(2,3)	0, 1	2
6	(1,5),(2,4)	2, 0	1
7	(1,6),(2,5),(3,4)	1, 2, 1	0
8	(1,7),(2,6),(3,5)	0, 1, 3	2
9	(1,8),(2,7),(3,6),(4,5)	2, 0, 0, 2	1

Games Q2 — Nim, pile of 9: first-move splits



Min plays first — (2,7) or (3,6) hand Max a losing (Grundy-0) position

Diagram — the 4 first-move splits of pile 9; teal boxes are the splits that hand Max a losing Grundy-0 position.

Because $g(9)=1 \neq 0$, the starting position is an N-position — the first player to move, **Min**, can force a win. Min's winning first moves are the splits whose two Grundy numbers are equal (XOR to 0): split 9 into (2,7) or (3,6). Any other first split, (1,8) or (4,5), leaves Max in a winning position.

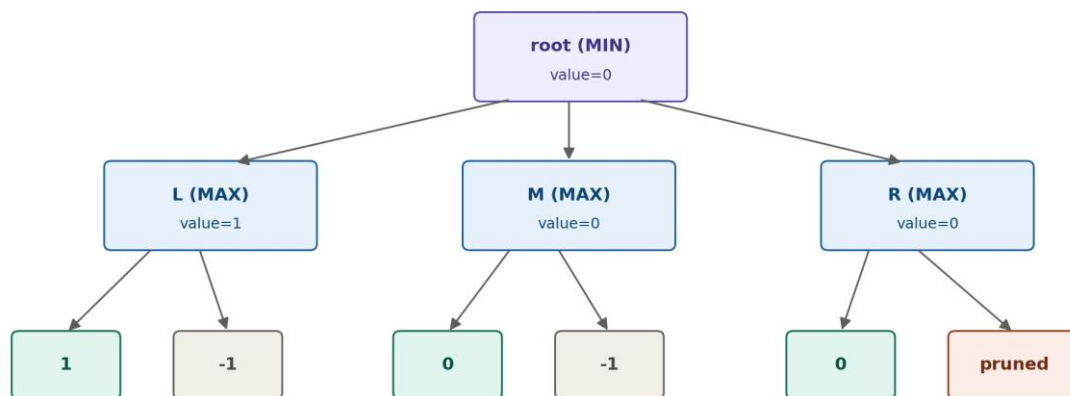
Question 3 — Minimax and Alpha-Beta on a tic-tac-toe subtree

Question: Fill in the internal minimax scores in the game tree; state Max's best move; then simulate alpha-beta pruning on the same tree.

Answer: Root is a MIN node (O to move) with three MAX children L, M, R. L has children L1 (value 1) and L2 (value -1). M has children M1 (value 0) and M2 (value -1). R has two terminal children, 0 and 1.

Node	Type	Children	Minimax value
L1	MIN (1 child)	1	1
L	MAX	L1=1, L2=-1	$\max(1, -1)=1$
M1	MIN (1 child)	0	0
M	MAX	M1=0, M2=-1	$\max(0, -1)=0$
R	MAX	0, 1	$\max(0, 1)=1$
Root	MIN	L=1, M=0, R=1	$\min(1, 0, 1)=0$

Games Q3 — Minimax + alpha-beta on tic-tac-toe subtree



Root = $\min(1, 0, \geq 0) = 0$. Under R, the 2nd leaf is beta-cut once Min already has 0 from M.

Diagram — minimax backup values, with the beta-cut prune under branch R shown in coral.

Root value = 0, achieved through M — Min (O) should choose the middle branch.

Alpha-beta (left to right): Visit L → returns 1, root's $\beta=1$. Visit M → returns 0, root's $\beta=0$. Visit R → first child gives $\alpha=0$; since $\alpha(0) \geq \beta(0)$, R's second child is **PRUNED** — Min already has a guaranteed 0 via M. Root = $\min(1, 0, \geq 0) = 0$, identical to full minimax, but one leaf was never evaluated.

Question 4 — Cryptarithmic CSP: $A+C=E$, $B+D=F$

Question: Variables A, B, C, D, E, F take values 1-7, all different, subject to $A+C=E$ and $B+D=F$. Solve by search tree. If $A=1$, find a solution.

Answer: With $A=1$, $E=C+1$. Try increasing C :

$C=2 \rightarrow E=3$. Leftover $\{4,5,6,7\}$: no two values sum to a third (min sum $4+5=9>7$) $\rightarrow$ dead end, backtrack.

$C=3 \rightarrow E=4$. Leftover $\{2,5,6,7\}$: $2+5=7 \checkmark \rightarrow B=2, D=5, F=7$. **Solution found.**

Games Q4 — Cryptarithmic search tree ($A+C=E$, $B+D=F$)

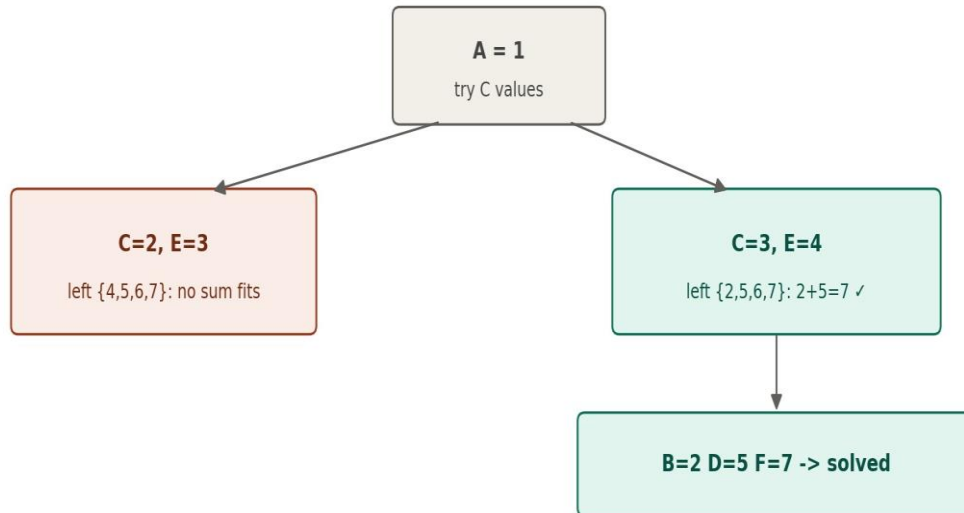


Diagram — branching on C : $C=2$ dead-ends, $C=3$ succeeds with $B=2, D=5, F=7$.

Solution: $A=1, C=3, E=4, B=2, D=5, F=7$ (unused value: 6). Not unique — continuing the tree also finds e.g. $C=5, E=6, B=3, D=4, F=7$ (unused value 2).

Question 5 — Alpha-beta pruning on a game tree

Question: Apply alpha-beta pruning to the given game tree; show where the alpha and beta cuts occur.

Answer: Example tree (MAX root, MIN one ply down), evaluated left to right:

Root's 1st child (MIN): leaves 3, 5, 2 $\rightarrow$ MIN=3. Root's α becomes 3.

Root's 2nd child (MIN): 1st leaf=2 $\leq \alpha(3) \rightarrow$ remaining leaves PRUNED (β -cut).

Root's 3rd child (MIN): leaves 6, 4 $\rightarrow$ MIN=4. Root's α updates to $\max(3,4)=4$.

Final root value = 4 (via the 3rd child). One full leaf and any leaves after the 2nd child's first leaf were pruned — a classic beta-cutoff.

Question 7 — 2×2 Tic-tac-toe with a “pass” move

Question: In 2×2 tic-tac-toe with a pass option (X first), draw the full game tree to depth 2, mark node values using $\text{eval} = \#X - \#O$, and circle any node alpha-beta (left-to-right) would not evaluate.

Answer: Every pair of the 4 squares forms a winning line, so marking any square is equivalent to any other by symmetry.

Depth-1 node values (MIN over O's replies): each "X plays sq.i" = $\min(0,0,0,1) = 0$. "X passes" = $\min(-1,-1,-1,-1,0) = -1$.

Root value = MAX over X's 5 options = $\max(0,0,0,0,-1) = 0$ — X should mark any square; passing is strictly worse.

Alpha-beta: "X plays sq.1" is explored fully first, setting root's $\alpha=0$. Every later branch only needs one child $\leq \alpha=0$ to prune the rest. In total, 13 of the 21 leaves are never evaluated — only the 4 leaves under X1 and the first child of each other branch actually get evaluated.

Part 3 — Quick Algorithm Reference

Algorithm	Frontier order	Complete?	Optimal?	Typical use
BFS	FIFO queue	Yes (finite b)	Yes, equal step costs	Shortest path, unweighted
DFS	LIFO stack	Yes (w/ visited set)	No	Memory-limited, CSP backtracking
UCS	priority by $g(n)$	Yes	Yes	Cheapest path (= Dijkstra)
Greedy	priority by $h(n)$	No	No	Fast but risky
A*	priority by $f=g+h$	Yes	Yes, if h admissible	Best general-purpose informed search
Minimax	full game-tree backup	Yes (finite tree)	Optimal vs perfect opponent	2-player zero-sum games
Alpha-Beta	minimax + α/β bounds	Yes	Same as minimax, faster	Same as minimax, deeper search